Slide 1: Code Review Agent

AI-Powered Automated Code Analysis

An Agentic AI Learning Project - ReACT Loop Architecture

Team: Person 1 Person 2 Person 3 Person 4 Person 5

January 2025

SPEAKER NOTES:

SPEAKER NOTES (1 min): Welcome everyone! Today we'll present our Code Review Agent - an AI-powered system that automatically analyzes code quality. Built over 2 weeks by a team of 5, this project demonstrates agentic AI using the ReACT loop pattern. We achieved an 85/100 audit score with 257 passing tests. Let's dive in!

Slide 2: Project Overview

What: AI system that autonomously reviews code from GitHub repositories Why: Learn Agentic AI development with ReACT loop architec

SPEAKER NOTES:

SPEAKER NOTES (1.5 min): This project was designed to solve real code review bottlenecks... We chose the ReACT (Reasoning + Acting) loop because it's the industry standard for autonomous AI agents. Key result: 85/100 audit score shows high code quality.

Slide 3: The Problem

Manual code reviews are time-consuming and inconsistent between reviewers Teams struggle to catch security vulnerabilities, perfor

SPEAKER NOTES:

SPEAKER NOTES (1 min): Manual code review is essential but expensive... A single review can take 30-60 minutes. Our agent automates the initial pass, catching common issues instantly.

Slide 4: Solution: Code Review Agent

End-to-End Automated Code Analysis Pipeline

GitHub URL User submits a repository URL for review

Agent Core ReACT loop processes code with LLM reasoning

Analysis Tools 5+ specialized tools analyze the codebase

Report Comprehensive review with scores & issues

Tech Stack: Python 3.10+ - FastAPI - Streamlit - Gemini AI - SQLAlchemy - Pytest

257 Tests - 85/100 Audit Score - 6,000+ Lines of Code - 2-Week Sprint

SPEAKER NOTES:

SPEAKER NOTES (1.5 min): Our solution is a complete pipeline from GitHub URL to comprehensive review report. The user submits a repository URL, our agent fetches the code, analyzes it using 5+ tools, and generates a detailed report with scores, issues found, and improvement suggestions.

Slide 5: What is the ReACT Loop?

Reasoning + Acting = Intelligent Agent Behavior

Think Agent analyzes context decides next action

Act Execute selected tool (run analysis, fetch data)

Observe Collect tool output & environment feedback

Reflect Process observations plan next iteration

Loop continues until task complete or max iterations reached

The ReACT loop enables the agent to reason about code, take actions like fetching files or running analysis, observe results, and

SPEAKER NOTES:

SPEAKER NOTES (2 min): Let me explain ReACT - it stands for Reasoning + Acting. Unlike traditional chatbots that just respond, ReACT agents actively think about what to do, take actions, observe results, and reflect on next steps. This is why modern AI agents work so well...

Slide 6: Key Components

1. Agent Core

ReACT loop controller - orchestrates Think Act Observe Reflect cycle Manages state, context window, iteration limits, and tool

2. Analysis Tools

5+ specialized tools: Code fetch, structure analysis, security audit, performance check, dependency analysis, quality scoring

3. LLM Integration

Gemini 2.0 Flash (primary) - OpenAI GPT-4o-mini (alternative) Handles reasoning, tool selection, report generation

4. FastAPI Backend

REST API endpoints for review submission, history, health checks SQLAlchemy + SQLite for persistence - async support

5. Streamlit UI

Web interface with code input, review history, live progress Sidebar configuration - real-time status updates

SPEAKER NOTES:

SPEAKER NOTES (2 min): The system has 5 main components working together... The Agent Core orchestrates everything. Tools give the agent capabilities. LLM provides reasoning. FastAPI serves the API. Streamlit provides the UI.

Slide 7: Technical Architecture

BACKEND (FastAPI Server)

API Layer routes.py, dependencies.py

Agent Core agent.py, agent_types.py

Analysis Tools code_fetch, security, performance, quality

LLM Integration gemini_client, openai_client, llm_client

Database SQLAlchemy + SQLite ReviewRecord model

Config & Utils settings.py, logger.py validation

FRONTEND (Streamlit)

UI Components streamlit_app.py

API Client api_client.py

TESTING (Pytest)

- 257 tests (unit + integration) - 85%+ code coverage - pytest-asyncio for async - Mock LLM provider for CI - SQLite in-memory for

SPEAKER NOTES:

SPEAKER NOTES (2 min): Here's the full architecture... Backend has 6 main areas: API, Agent, Tools, LLM, Database, and Config. Frontend is Streamlit with API client. Testing has 257 tests with 85%+ coverage using mock providers.

Slide 8: Team Contributions

Person 1 Agent Architect

Designed ReACT loop architecture Implemented agent core, context management State machine, iteration control

Person 2 Tool Engineer

Built 5+ analysis tools Code fetch, security audit, performance File structure analyzer, quality scorer

Person 3 LLM Specialist

Gemini 2.0 Flash integration OpenAI provider implementation Prompt engineering, response parsing

Person 4 Backend Developer

FastAPI routes, dependencies Database models, SQLAlchemy Configuration, logging utilities

Person 5 Frontend Developer

Streamlit UI with 4 screens API client integration Progress tracking, history view

SPEAKER NOTES:

SPEAKER NOTES (2 min): Each team member owned a critical component... Person 1 built the ReACT loop, Person 2 created analysis tools, Person 3 handled LLM integration, Person 4 built the backend API, Person 5 created the Streamlit frontend.

Slide 9: Features Implemented

Features Implemented

ReACT Loop - Autonomous reasoning with Think Act Observe Reflect cycle 5+ Code Analysis Tools - Fetch, structure analysis, sec

SPEAKER NOTES:

SPEAKER NOTES (1.5 min): Let me walk through what we actually built... The ReACT loop is the core innovation. We have 5 specialized tools. The system supports multiple LLM providers. Full API documentation is auto-generated by FastAPI. And we validated everything with 257 tests.

Slide 10: Technology Stack

Backend

Python 3.10+ - Core language

FastAPI - REST API framework

SQLAlchemy - ORM with SQLite

Uvicorn - ASGI server

Frontend

Streamlit - Web UI framework

Requests - API client library

Markdown - Rich text rendering

AI / LLM

Gemini 2.0 Flash - Primary LLM

OpenAI GPT-4o-mini - Alternative

Google Generative AI SDK

Custom prompt engineering

Testing & Quality

Pytest - Test framework

pytest-asyncio - Async tests

pytest-cov - Coverage reports

Ruff - Linter & formatter

MyPy - Type checking

SPEAKER NOTES:

SPEAKER NOTES (1 min): Our tech stack is modern Python ecosystem... FastAPI for backend, Streamlit for frontend, Gemini for AI reasoning. We also have comprehensive testing infrastructure with Ruff linting and MyPy type checking.

Slide 11: Project Statistics

6,000+

Lines of Code

27

Source Files

257

Tests Passing

85+%

Code Coverage

85/100

Audit Score

2 Weeks

Development

SPEAKER NOTES:

SPEAKER NOTES (1 min): The numbers speak for themselves... 6,000+ lines of production Python code, 257 tests all passing, 85%+ code coverage, and an 85/100 audit score from our internal code review process.

Slide 12: How It Works - Demo Flow

Step 1: Submit URL

Paste GitHub repo URL in UI

Step 2: Agent Starts

ReACT loop initializes context

Step 3: Fetch Code

Download & analyze structure

Step 4: Run Tools

Security, perf, quality checks

Step 5: Generate

LLM creates review report

Step 6: Display

Results shown in Streamlit UI

Total time: ~30-60 seconds per review - Results include: quality score, issues found, security vulnerabilities, performance bottle

SPEAKER NOTES:

SPEAKER NOTES (1.5 min): Let me walk through the demo flow... User submits a GitHub URL. Agent starts the ReACT loop. It fetches the code, runs analysis tools, generates a report using LLM, and displays results. The whole process takes 30-60 seconds.

Slide 13: Results & Demo

[Streamlit UI Screenshot]

[API Docs Screenshot]

[Review Results Screenshot]

[Test Coverage Screenshot]

SPEAKER NOTES:

SPEAKER NOTES (1.5 min): Here are screenshots of the working system... The Streamlit UI is clean and professional. FastAPI automatically generates API docs. Review results show quality scores with detailed issue breakdown. Our test coverage is 85%+ across all modules.

Slide 14: Lessons Learned

ReACT framework is powerful but prompt engineering is critical for quality results Tool design matters - well-designed tools make

SPEAKER NOTES:

SPEAKER NOTES (1.5 min): We learned a lot building this... The ReACT framework is powerful but the quality depends heavily on how you prompt the LLM. Well-designed tools make the agent smarter. Testing caught many regressions. Modular code let us work in parallel.

Slide 15: Future Improvements & Conclusion

Future Improvements

Multi-language support (Java, C++, Go)

Custom rule definitions for teams

Automated PR comments & fix suggestions

CI/CD pipeline integration

Real-time collaborative reviews

Conclusion

Successfully built an agentic AI code review system

ReACT loop architecture proven effective

85/100 audit score validates code quality

257 tests ensure reliability

Team collaborated effectively across all roles

SPEAKER NOTES:

SPEAKER NOTES (1.5 min): Looking ahead, we could add multi-language support, custom rules, and CI/CD integration. But even in its current state, the system works well - 85/100 audit score, 257 tests, and a successful team collaboration across all roles.

Slide 16: Questions & Discussion

Thank you for your time!

We welcome your questions and feedback

Contact: code-review-agent@project.com

Repository: github.com/code-review-agent

Slides available at: presentations/Code-Review-Agent-Overview.pptx

SPEAKER NOTES:

SPEAKER NOTES (1 min): Thank you everyone for listening! We're happy to answer any questions about the architecture, implementation decisions, or how we'd extend this further.